Introduction

The Hierarchical Reasoning Model (HRM) by Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changeling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori, all from Sapient Intelligence, is an alternative architecture to the Chain-of-Thought (CoT) transformer for complicated reasoning tasks. Mimicking how the human brain operates over different amounts of information at different timescales, the architecture has both high and low level components (both implemented as encoder-only transformers), where the low level model is run multiple times for every pass through the high level module. These modules only contain the transformer blocks of the transformer, not the embedding or head layers, causing the model to only discretize the hidden state to token space as the very last step of the model, allowing for more information to propagate between layers. This also makes it that attention within these layers operates on vectors of the same dimensionality at each step, meaning that compute grows linearly with the amount of 'thinking steps' required to solve a problem rather than quadratically as in a CoT Transformer. The architecture also takes gradient steps after every few high level steps (called a segment), rather than only at the final prediction. We believe that this forces the modules to exhibit local hidden-state contraction to fixed-points, allowing for gradient-approximation optimizations and iterative solution refinement. Finally, Wang et al has empirically shown, and we have confirmed, that this architecture performs significantly better on tasks such as Sudoku than SOTA architectures orders of magnitude larger.

Chosen Result

Our goal was to validate the performance of the HRM on Sudoku. Sudoku is an interesting problem because it requires a complicated[1] chain of thought that is nonetheless linear and deterministic. In addition, the paper's result for Sudoku showed stark contrast between HRM and the CoT baselines, proving its benefit. Beyond the paper's primary focus, there were also a few open questions and unjustified claims we wanted to address. In particular, we wanted to empirically show the local fixed-point contraction (an assumption barely justified in the paper), and determine if model size can be increased after training (implied by the architecture but not mentioned in the paper).

Methodology

The Hierarchical Reasoning Model (**Figure A**) can be thought of as a mix between a transformer and a recurrent model. HRM uses 2 encoder-only transformer modules: an L-module that processes frequently, and an H-module that processes less frequently. The L-module handles lower-level updates, while the H-module integrates information over a longer timescale. Unlike a standard transformer that maps to token space after each layer, HRM reasons entirely through its hidden state and only converts its final representation into token predictions through the output head. The repeated segments of the model are identical, as the respective L-modules and H-modules share the same weights. In our notation, M is the segment count, N is the number of H-module updates per segment, and T is the L-to-H update ratio. During training, the model uses deep supervision, meaning that the prediction head is applied and a gradient step is taken multiple times per forward pass (once at the end of each segment). To make this feasible without fully backpropagating through all recurrent steps, the paper uses a one-step gradient approximation made possible by the theorized fixed-point convergence, which we also use in our reimplementation. Finally, the paper introduces a version of Adaptive Computation Time (ACT), which lets the model stop early during inference instead of running for a fixed segment count. With ACT, the model learns a halting decision at each segment, allowing easier Sudoku boards to use less computation while harder boards can continue reasoning up to an M_{max}.

We trained the HRM on the HRM-checkpoint-sudoku-extreme dataset used in the paper, sampling 2^{18} of the ~3M puzzle examples for training. Sampling a subset of data makes training more feasible with limited compute while still having enough data to achieve strong generalization. The model has 33.6M parameters, determined by the fixed HRM hyperparameters, and was trained for 328k steps, fewer than the paper due to compute constraints. Our training procedure used an initial learning rate of 1e-4 with 5% linear warmup, followed by cosine decay to 1e-5[2]. Unlike the paper, we used AdamW rather than its Adam variant due to familiarity and added 20% dropout to combat possible overfitting due to a smaller dataset size. To test the paper's claims regarding ACT's computational efficiency, we compared it against the Fixed-M method at different M_{max} values. Additionally, we implemented & trained an Encoder-Only Transformer and Bidirectional LSTM on the Sudoku data to serve as a baseline, using roughly the same number of parameters for a fair comparison.

---

[1] Sudoku is NP-complete if the size of the board is $n^2 \times n^2$.

[2] This is different from the scheduler named in the paper, but it matches the one used on HRM's official GitHub.

Results & Analysis

The re-implementation approaches the token accuracy of the paper benchmark result, indicating that the model we created was capable of matching their results, even having only trained on a fraction of the dataset. With M=16 & T=2, token accuracy reached 92.12%, and with M=128 & T=8 it reached 98.04% (**Figure B**). Compared to the paper's 99.10% token accuracy with M=8, we were able to take advantage of longer inference to close most of the gap with the paper's results, even with a smaller model and dataset size. Our loss plot supports this notion, as the loss converges to a low minimum and generalizes well (**Figure C**). In contrast, observing the loss plots, it is clear that both the BiLSTM and Encoder-Only Transformer models experience overfitting when using the same training set size as HRM (**Figure D, E**). Performing the same test with 8x more training examples removed the overfitting issue, but we observed in the loss curves that both baseline models suffer from slow convergence and poor generalization (70.37% token accuracy for Transformer, 52.85% for BiLSTM, **Figure F, G**). The structure and behavior of the HRM clearly allows it to empirically perform better at complex reasoning tasks.

Benchmarking ACT showed that Fixed-M inference scales linearly with M_{max}, while ACT grows logarithmically by halting once enough computation has been used (**Figure H**). This supports the paper's claim that ACT lets HRM adapt its inference depth to the difficulty of each Sudoku board, stopping early on easier examples without meaningfully reducing accuracy.

We extended our exploration beyond benchmarking accuracy with the intent to test some of the paper's unaddressed architectural assumptions. We first wanted to look at whether the hidden state contracts to a fixed point corresponding to a stable answer. From our forward residual plot, we observed that the hidden state changes substantially early on, but stabilizes as soon as the model reaches the solved board (**Figure I**). This implies that the fixed-point assumption made by the paper is accurate at least upon reaching a hidden state corresponding to a Sudoku solution. The validity of this assumption before reaching a solved hidden-state, and additional related assumptions (such as those motivating the one-step gradient) require further exploration.

Additionally, we looked to test if the shared segment structure lets us increase inference depth after training. We found that increasing M and T at inference-time greatly improves accuracy in a clear trend, exceeding 98.09% token accuracy at M=128 & T=8 (**Figure J**). This shows that HRM can be scaled to improve performance after training and beyond its train-time inference depth. This particular result shows that while Sudoku is a controlled problem we used for consistent benchmarking, the HRM style of model allows for post-training scaling that could be applied to other structured reasoning problems where a model can iteratively refine an internal solution before producing an answer. This is an extremely important result for the Machine Learning discipline, and also requires further exploration.

Reflections

There are two important observations we made over the course of our project. We learned that (1) new architectures can still solve many problems far better than more well-known architectures at the same scale, and (2) all parts of the design of an architecture, from when gradient steps are taken to how blocks are organized, matter in determining emergent properties. Additionally, there are two re-implementation takeaways that we left with: (1) the HRM architecture vastly outperformed SOTA models, even when comparatively underparameterized, converging faster and generalizing better than both a standard encoder-only transformer and a bidirectional LSTM, and (2) this architecture allows for increasing the model size post-training through both increasing the number of segments and the L-module:H-module ratio, leading to notably increased performance.

In addition, we are interested in exploring four directions in future research. First, a function must be locally contractive to approach a stable fixed point. We would like to gauge the spectral radius of a segment in a trained model through power iteration instead of our current approach of analyzing the forward residuals of each step. Second, in order for the one-step gradient approximation used by the HRM to be valid, the spectral radius of the Jacobian of a segment must be much, much less than one. This can be tested during training, and is completely unjustified by the paper. Third, while the paper's ACT is clearly powerful, it adds a significant amount of parameters to train. As we observe clear patterns in the forward residual graphs for each Sudoku– that is, as soon as the model gets the answer, the residual drops to near zero– it alone could be a meaningful heuristic for when to stop. Fourth, and most importantly, we want to elaborate on our discovered property that expanding the network post-training leads to increased performance. We think that this could be incredibly useful in the ML scale-up regime that currently exists, even if it only applies to non-autoregressive applications.

Figure A:

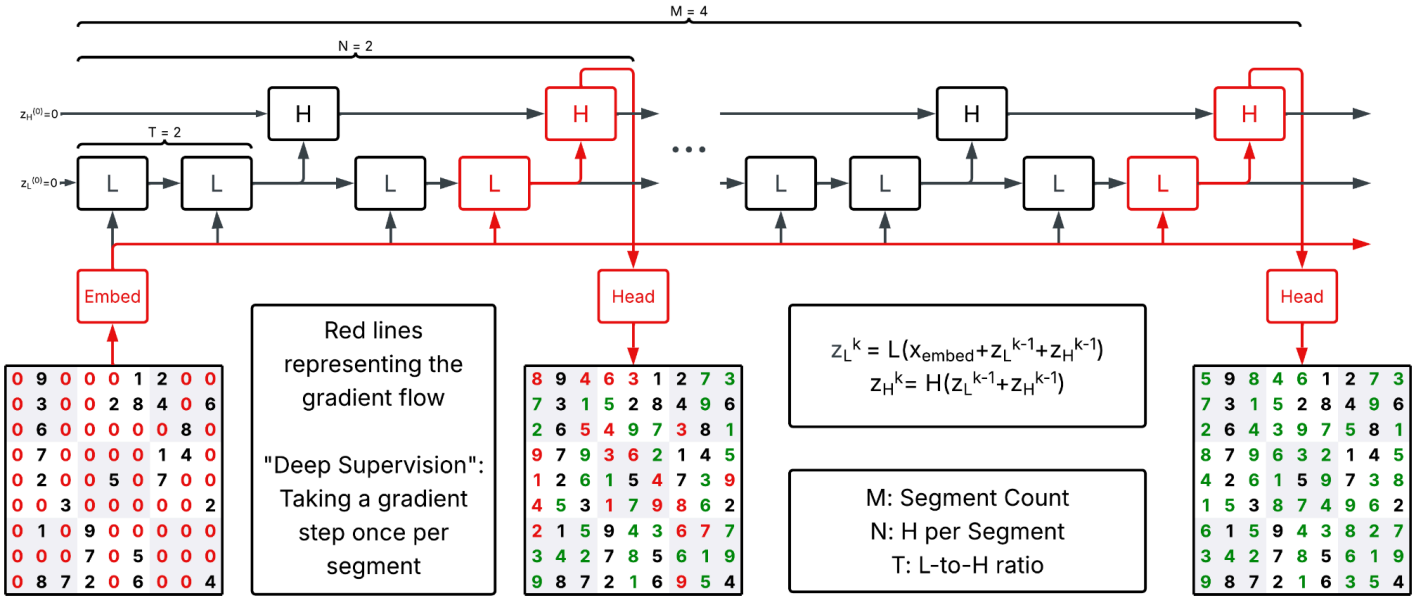


Figure B:

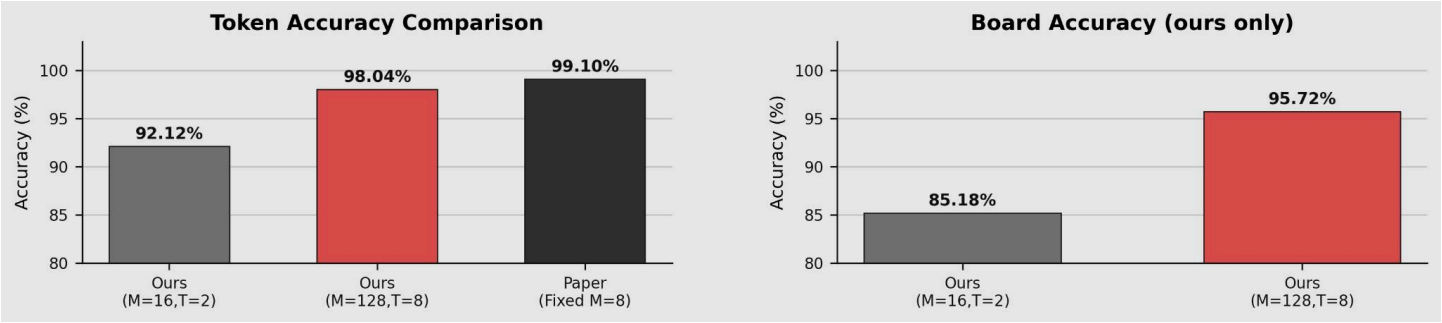


Figure C:

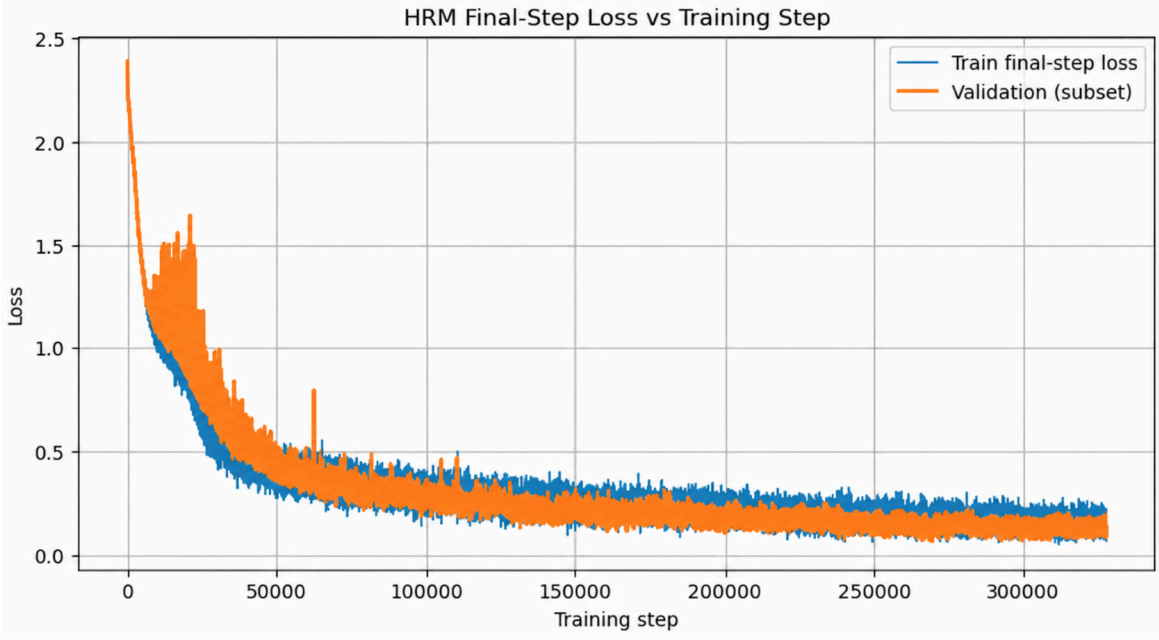


Figure D:

Figure E:

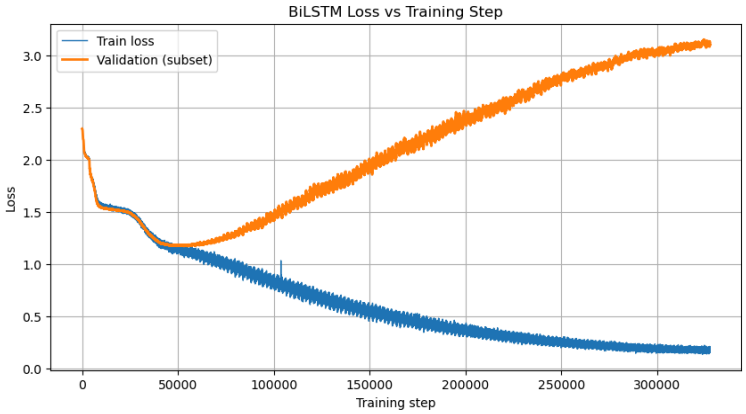


Figure F:

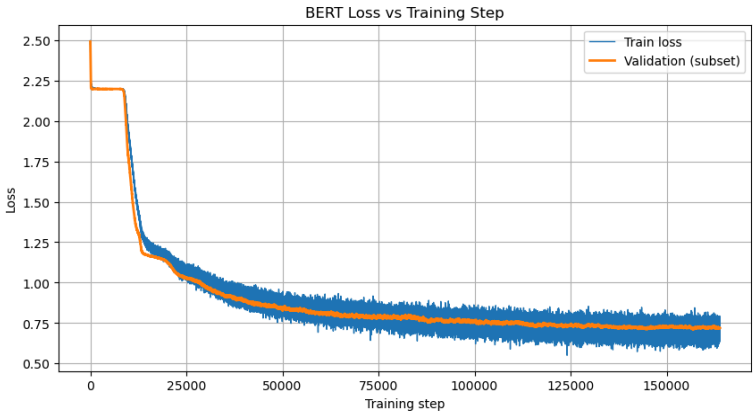


Figure G:

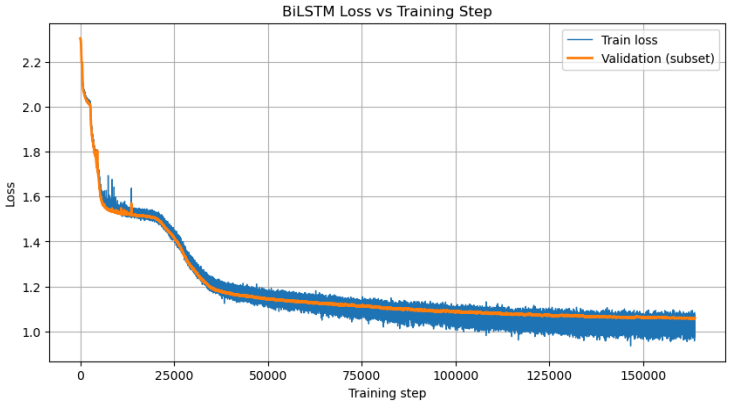


Figure H:

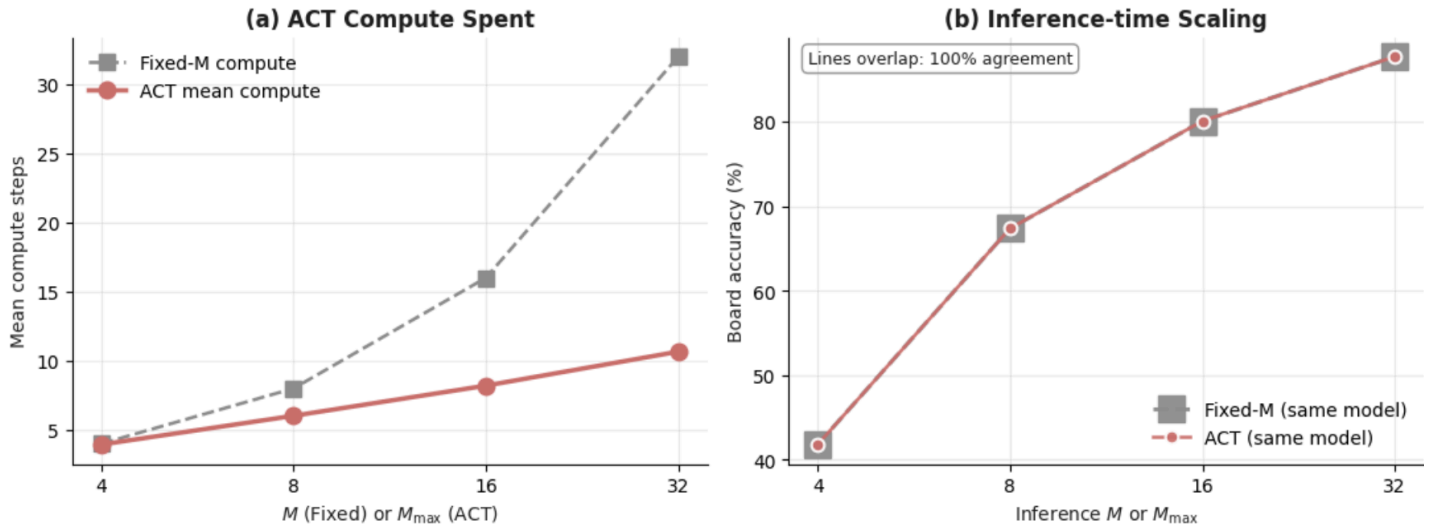


Figure I:

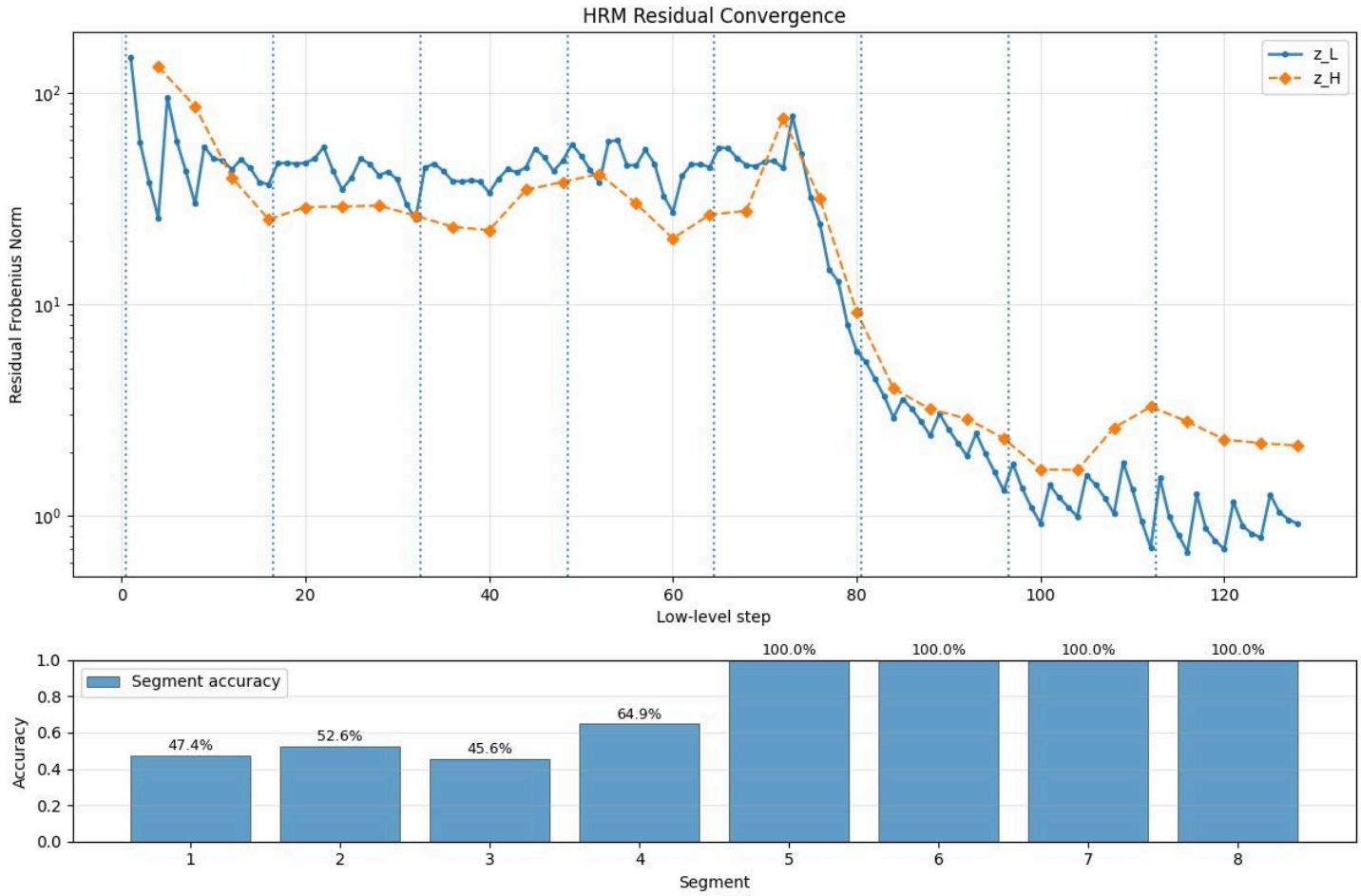
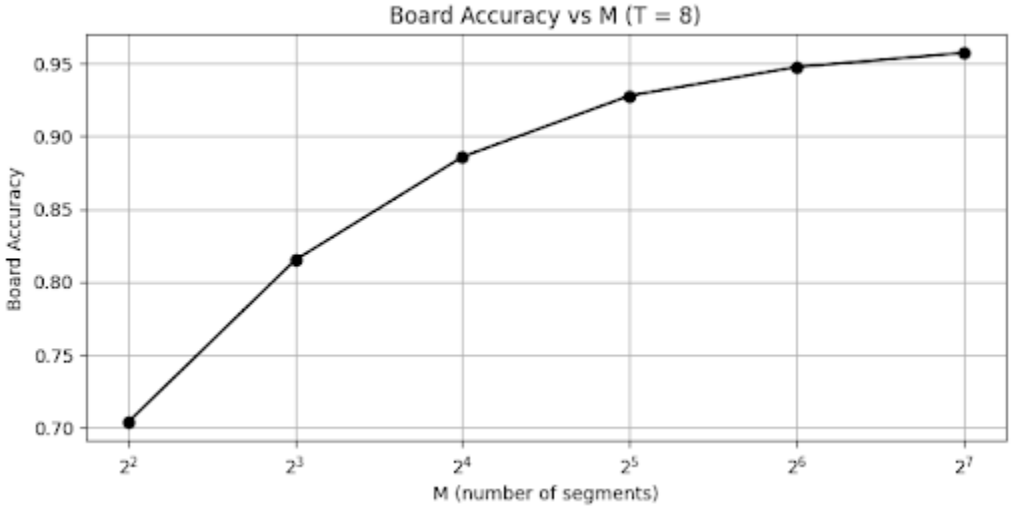


Figure J:



References

Wang et al., *arXiv:2506.21734* (2025),
D. Kahneman, *Thinking, Fast and Slow* (2011),
Sapient Intelligence, s*udoku-extreme* (2025),
Paszke et al., *arXiv:1912.01703* (2019),
Hunter, *Matplotlib: A 2D Graphics Environment* (2007),
da Costa-Luis, *tqdm: A Fast, Extensible Progress Meter for Python and CLI* (2019),
Lhoest et al., *Datasets: A Community Library for Natural Language Processing* (2021),
Hugging Face, *huggingface-hub* (2026),
Python Software Foundation, *Python 3.13 Documentation* (2024).

GitHub Repo Link: https://github.com/Eyiteus/HRM_Reconstruction